COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

SYNTHESIS OF SOUND FOR APPLICATION IN AN INTERACTIVE GAME FOR VISUALLY IMPAIRED USERS

Master thesis

2025 Bc. Norbert Hašan

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

SYNTHESIS OF SOUND FOR APPLICATION IN AN INTERACTIVE GAME FOR VISUALLY IMPAIRED USERS

Master thesis

Study program:	Applied informatics
Branch of study:	Applied informatics
Department:	Department of Applied Informatics
Supervisor:	Mgr. Lukáš Gajdošech
Consultant:	

Bratislava, 2025

Bc. Norbert Hašan

67751524

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Norbert Hašan
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Syntéza zvuku pre použitie v interaktívnej hre určenej pre ľudí so zrakovým znevýhodnením
Synthesis of sound for application in an interactive game for visually impaired users.

Anotácia: Ľudia so zrakovým znevýhodnením vyžadujú pri hraní interaktívnej hry väčšie množstvo informácie sprostredkovanej pomocou zvuku. Vďaka takémuto výstupu sa vedia napr. zorientovať a vyriešiť priestorový hlavolam. Produkcia zvukovo bohatých prostredí je časovo náročná, vyžaduje zber veľkého počtu zvukových vzoriek a ich následné spracovanie. Toto spracovanie je pritom často pomerne dobre zautomazivateľné, napr. pridanie ekvalizéru, alebo využíva randomizáciu parametrov na vytvorenie viacerých variácií. Príkladom môže byť napr. vytvorenie zvuku pre kroky na rôznom podklade (hlina, piesok, dlažba atď.). Predmetom tejto práce je preskúmanie a návrh rôznych prístupov syntézy zvuky na základe textového popisu dizajnéra. Metódy zahŕňajú analytické ako aj moderné prístupy založené na generatívnych neurónových sieťach.

Cieľ:
- preskúmanie existujúcich metód syntézi zvuku
- konzultácia s cieľovou skupinou používateľov, špecifikácia ich požiadaviek
- vytvorenie modelu na automatickú generáciu zvukových efektov na základe textového popisu

Literatúra:

- MetaSounds. In: Unreal Engine Documentation. https://docs.unrealengine.com/5.3/en-US/metasounds-in-unreal-engine/
- Barahona-Ríos, Adrián and Tom Collins. SpecSinGAN: Sound Effect Variation Synthesis Using Single-Image GANs. In: Proceedings of the 19th Sound & Music Computing Conference, Saint-Étienne, France, 2022.
- Andreu, S., & Villanueva Aylagas, M. Neural Synthesis of Sound Effects Using Flow-Based Deep Generative Models. In: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 18(1), 2-9. 2022. https://doi.org/10.1609/aiide.v18i1.21941

Kľúčové slová: zvuková syntéza, audio hra, generatívne neurónové siete

Vedúci: Mgr. Lukáš Gajdošech
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

I hereby declare that I have written this thesis by myself, only with help of referenced literature, under the careful supervision of my thesis advisor.

. . . . . .

Bratislava, 2025 Bc. Norbert Hašan

Acknowledgement

Abstract

Keywords:

Abstrakt

Kľúčové slová:

Contents

Introduction **1**

1 Motivation **3**

1.1 The Core Problem 3

1.2 The Limitations 4

1.3 The Proposed Solution 4

1.4 Primary Objectives 5

2 Theoretical Background **6**

2.1 Fundamentals of Digital Audio Signal Processing 6

2.1.1 Discrete-Time Signals and Sampling Considerations 6

2.1.2 Short-Time Fourier Transform (STFT) 7

2.1.3 The Mel-Scale 8

2.1.4 Normalization and Log-Amplitude Representation 9

2.2 Machine Learning & Deep Generative Models 10

2.2.1 Foundations of Neural Networks 10

2.2.2 Taxonomy of Generative Architectures 12

2.2.3 The Generative Adversarial Network Framework 12

2.2.4 Advanced Architectural Components for Audio 13

2.3 GAN Stability 14

2.3.1 GAN Training 14

2.3.2 Hinge Loss 14

2.3.3 Spectral Normalization 15

2.3.4 Feature Matching Loss 16

2.4 Natural Language Conditioning 16

2.4.1 Contrastive Language-Audio Pretraining (CLAP) 17

2.4.2 Conditional Group Normalization (CGN) 17

2.4.3 Projection Discriminator 18

2.5 Neural Vocoding 19

2.5.1 Phase Reconstruction 19

2.5.2 Generative Vocoding 19

2.5.3 NVIDIA BigVGAN 20
2.6 Evaluation Metrics for Generative Models 20
2.6.1 From Inception Distance to Fréchet Audio Distance 21
2.6.2 Subjective Evaluation 21

3 Related Work **23**
3.1 Procedural Game Audio 23
3.2 Autoregressive Audio Generation 24

4 Work methodology and Research methods **26**

5 Implementation **27**
5.1 Architecture Design 27
5.1.1 The General network (G) 27
5.1.2 The Critic Network (D) and Wasserstein Loss 28
5.1.3 Stabilization via Gradient Penalty 29

6 Research **31**

7 Results and Discussion **32**

List of Figures

1.1 Sound creation pipeline in Blind Octopus Studio. 4

2.1 Speech in a spectrogram 8
2.2 A graph illustrating the Mel filter bank 9
2.3 Rectified Linear Unit compared to Leaky Rectified Linear Unit 11
2.4 A classic GAN architecture diagram. 13
2.5 Hinge loss compared to cross entropy loss. 15
2.6 Architectural flow of CGN. Text embedding c is projected to predict the affine parameters γ and β. This modulates normalized audio feature maps. 18
2.7 Enter Caption 21

3.1 Comparison of sample-based methodologies with procedural methods . 24

List of Tables

Terminology

Introduction

Focus of this work is to develop a pipeline that automates synthesis of sound for interactive applications. Exploring the generation of class specific sounds effects, that are generated based on a textual description using generative neural network. Sounds like these can be used to generate audio environments. This thesis specifically looks onto the possibility to reduce the the time consuming job of manual sound design [6]. The idea to tackle this is to build a Generative Adversarial Network (GAN) [14] that operates on mel-spectrograms. Comparing this custom architecture to generative models that operate on text-to-audio approach like AudioLDM [29], AudioGen [24], Riffusion[1][2], but also to non textual single image GAN approaches(SpecSinGAN) [3], allows to evaluate viability.

While this project is heavily inspired by the specific requirements of visually impaired gamers [12], the resulting generative models are broadly applicable to the game development overall. The following is an introduction to the objectives and sections of this thesis.

In chapter one, the motivation behind the work is explained. The reason why it is difficult to produce sound-rich environment, especially for accessibility purposes. And how machine learning tools can make the workflow easier.

fill in the
update b
on the te
sections!!

The second chapter focuses on explaining the theoretical information that is needed while reading this work. It goes over digital audio processing, with primary focus on extraction and reconstruction of mel-spectrograms. The second chapter also explains machine learning principles, like GAN architectures, diffusion models and integration of language models for text conditioning like CLAP [9] or BERT [8].

Chapter three reviews already existing methods of sound synthesis. It introduces architectures of current models utilized in research. Comparison of established models spanning from simple procedural tools [11] to different neural synthesis approaches [1].

In the fourth chapter, specialization of this work is presented, with the insights gathered from users, with focus on visually impaired users. Also the goal contribution of this thesis is presented along with functional goals for automatic generation model.

Chapter five represents the core of this thesis, diving deeply into the methodologies

[1] https://riffusion.com/about
[2] github.com/hmartiro/riffusion-inference

and experimentation used during the work. At first detailing the design, preparation of the dataset and training of the GAN, while focusing on the conversion pipeline between audio signals and mel-spectrograms. At the end documenting the practical research of existing solutions.

Chapter six assesses the developed methods. Contains comparison between generated audio and baselines using quantitative metrics. Alongside that, shows results from testing with user group containing visually impaired.

The closing chapter focuses on conclusion, reviews satisfaction of the goals and discusses possibilities for future work and possible optimization for the systems architecture.

Chapter 1

Motivation

Historically, in video games or other types of interactive media, the visual aspect had always received more attention. Prioritizing computational resources to picture perfect like graphics. Equally critical for immersion, yet often very under optimized is the auditory environment. One of the main motivations of this thesis comes from specific accessibility requirements for visually impaired users, the practical bottlenecks of sound design industry and the methodological gap between visual and audio related machine learning. Taking a look at the present limitations in production of manual audio and applying novel generative approaches it is possible to build an automated, text driven sound synthesis pipeline [24, 29].

1.1 The Core Problem

In typical game development, audio is mostly just an addition to the visuals. However, when designing interactive applications specifically for visually impaired users, this has to be entirely inverted. In an environment of an audio game, the sound is no more just an aesthetic. It becomes a primary interface for the user. It becomes a compass navigating them through the game itself. Even outside of the game. Thanks to auditory cues visually impaired players are able to orient themselves, solve puzzles and perceive the surrounding setting.

For such a player a sound of a footstep means much more than just a sound effect. It communicates texture of the ground, their proximity to a obstacle and the size of their surroundings. Because such player extracts so much more from a sound, they are naturally more sensitive to audio repetition. If a player walks around dirt path in a game for ten minutes, they will not mind hearing the same footstep sample in a randomized order, but for the visually impaired gamer it becomes tiring fast. To crate accessible environment that feels real, developers must use large volume of audio variations.

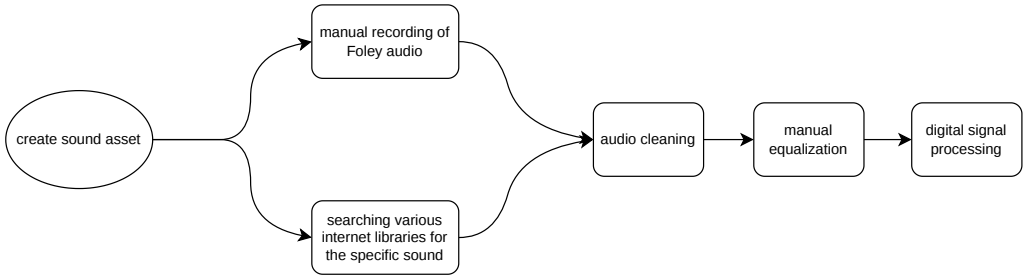

Figure 1.1: Sound creation pipeline in Blind Octopus Studio.

In reality, this requirement is rarely met. In Blind Octopus Studio[1], creation of sound assets is manual. Traditional pipeline of creating a simple audio file can be seen in the image1.1. When a game needs hundreds variations of single interaction type, the manual labor behind that asset creates both developmental and financial bottleneck.

1.2 The Limitations

Sound designers cannot produce infinite variations of a sound manually. Especially not for a dynamic acoustic environment. In the past, there were multiple efforts to bypass this using procedural audio. Although this approach may be computationally cheap [11], it quickly becomes too artificial.

In the present machine learning seems to offer a potential solution, but the progress seems to be asymmetrical. While visual generation is experiencing renaissance, where both Generative Adversarial Networks [14] and diffusion models [19] designed for image synthesis are capable of photorealistic fidelity [4, 20], the generation of audio using these methods lags behind.

Notoriously, synthesizing raw audio waveforms directly is difficult for neural networks. This is due to extreme density and complexity of the data. Due to that, training a network that maintains long term structural coherence over thousands of samples proves to be of huge difficulty. AudioLDM [29] and AudioGen [24] produce good results, but on the cost of exhausting computations. Such process consist of iterative denoising of signal over hundreds of steps [19]. This makes them too slow for the fast game development workflows.

1.3 The Proposed Solution

The mentioned discrepancy between visual and auditory AI presents an opportunity. If the approach of image generating GANs is used for audio generation, the challenge of synthesizing audio can be partially reframed to an image generation problem [3].

[1]Slovak Indie Game studio, https://blindoctopus.studio/

To bridge the gap between efficiency and generative fidelity, sound is treated as visual medium. By applying Short-Time Fourier Transform [15], it is possible to convert 1-dimensional temporal audio signals into 2-dimensional image representations known as mel-spectrograms. This in theory allows the GAN to avoid the issues with generative audio, and use the much more optimized mechanics of image generation. Through textual input it generates a mel-spectrogram, this synthetic image is then reconstructed back into audible waveform [15].

Ideally, the idea of the solution extends further than just accessibility context. The phenomenon of the "Curb-Cut Effect" [5], features designed specifically for individuals with disabilities inevitably benefit the broader population. The motivation originates from the needs of visually impaired gamers, the result is universally applicable.

1.4 Primary Objectives

- Review and analyze the technical distinctions between procedural audio [11], flow based models [1], single image GANs[3] and diffusion models [19, 29].
- Consult with the targeted audience of visually impaired users, get data for their specific acoustic requirements, spatial orientation needs.
- Design, build and train custom Generative Adversarial Network that can prove that it is possible to produce sound effects from natural language textual descriptions through generation of mel spectrograms.
- Compare the performance, inference speed and audio quality to other used architectures.

Chapter 2

Theoretical Background

2.1 Fundamentals of Digital Audio Signal Processing

The implementation of a generative model for environmental sound synthesis especially for textures such as footsteps on gravel requires a rigorous definition of the data representation. While sound physically exists as continuous air pressure fluctuations, deep generative models require discrete, structured data. This section outlines the signal processing pipeline, justifying the specific discretization parameters required to prepare raw audio for the extraction of neural features [3].

2.1.1 Discrete-Time Signals and Sampling Considerations

The first stage of the pipeline is the discretization of continuous audio signals, denoted as $x(t)$ into discrete sequences $x[n]$. This process is governed by the Nyquist-Shannon Sampling Theorem[20][33]. This theorem dictates that the sampling frequency f_s must be at least twice the maximum frequency component f_{max} of the signal to prevent aliasing artifacts ($f_s \geq 2f_{max}$). The Nyquist limit is theoretically the maximum reproducible frequency at a given sampling rate.

In the domain of neural audio synthesis, the choice of f_s is a critical hyperparameter that balances acoustic fidelity against computational dimensionality and memory overhead [6].

- **High Fidelity:** Works focused on high-resolution texturing, such as SpecSinGAN, often utilize a standard rate of 44.1 kHz to capture the full spectral brightness of transients [3]. Although this can capture the maximum spectral brightness of transients, the data vectors are extremely dense. This exponentially increases the memory footprint during the model training phase.

- **Efficiency:** Flow-based models like `WaveFlow` (as adapted by Andreu & Aylagas) often downsample to 16 kHz. This enables faster iteration and stable training

on consumer hardware[1]. Significantly reducing the dimensionality comes at a cost of discarding all acoustic information above 8 kHz, making the sounds a bit muffled.

For the theoretical scope of environmental sound synthesis, an intermediate sampling rate (such as 22,050 Hz) is often chosen as an optimal middle ground. This creates a "Nyquist ceiling" of approximately 11 kHz. Ideal for environmental sound synthesis for these two reasons:

- Most of the semantic texture and transient energy, that is necessary for identification of impact sounds like footsteps, or wood knocks falls within the lower 10 kHz band. By halving the standard rate for a CD, 44.1 kHz the input dimensionality is also reduced by 50%. This allows for bigger batch sizes during training of Generative Adversarial Network (GAN). Also, unlike the 16 kHz models the higher frequencies should not be degraded.

- State-of-the-art neural vocoders required for the final phase reconstruction, such as the NVIDIA BigVGAN architecture [**?**], offer highly optimized pre-trained weights specifically engineered for 22 kHz audio. Aligning the dataset sampling rate to 22 kHz provides the opportunity to avoid upsampling or downsampling during inference, which would be destructive.

2.1.2 Short-Time Fourier Transform (STFT)

To extract structured frequency features from the discrete 1D audio sequence $x[n]$, the signal must first be transformed into a 2D time-frequency representation fig. 2.1. To achieve this representation Short-Time Fourier Transform (STFT) is performed over the 1D audio sequence. STFT computes the Discrete Fourier Transform (DFT) over short, overlapping segments (windows) of the signal, isolating the frequency content at specific moments in time [15].

STFT mathematical formulation:

$$X(m, \omega) = \sum_{n=-\infty}^{\infty} x[n] \cdot w[n - mH] \cdot e^{-j\omega n} \tag{2.1}$$

where the w is the windowing function and H is the hop length.

In this thesis where the aim is to extract environmental audio features, these parameters need to be strictly defined:

- **FFT Size (N_{FFT}):** Defines the frequency resolution. A larger window allows the capture of the harmonic resonance of environmental materials (e.g., metal vs. concrete).

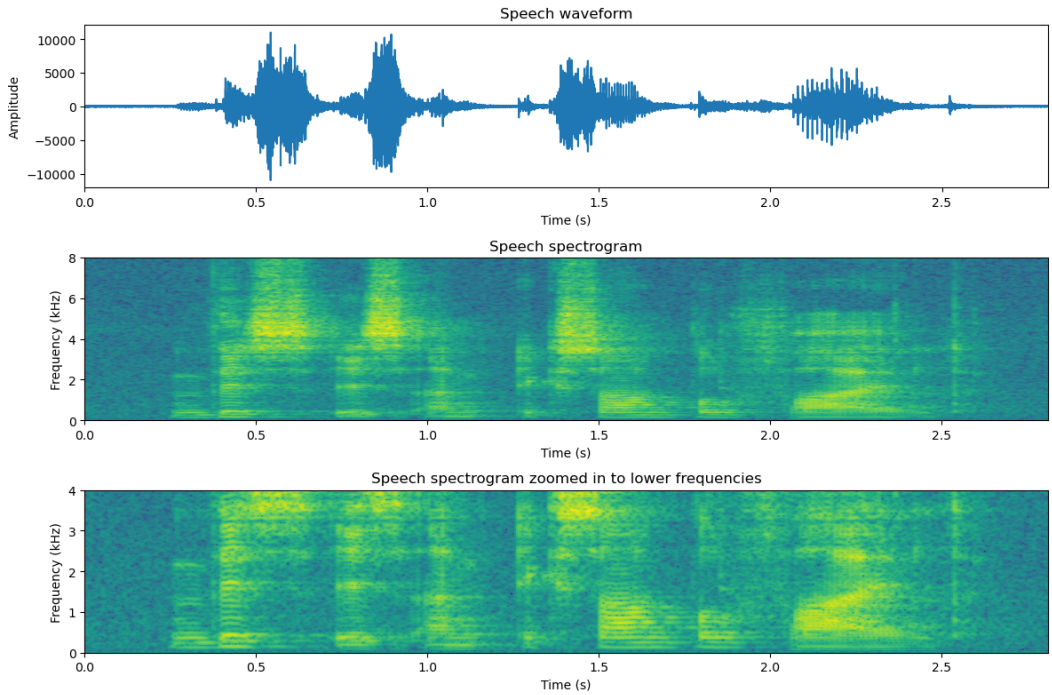

Figure 2.1: Speech in a spectrogram

- **Hop Length (H):** Defines the step size between windows, dictating the temporal resolution.

A high degree of window overlap (like 75%), ensures high temporal resolution. This is crucial to accurately model impact sounds like footsteps or gunshots, where the acoustic energy decays within milliseconds [3]. The SpecSinGAN study found out $N_{FFT} = 512$ to be most consistent, but unable to capture resonance well.

2.1.3 The Mel-Scale

The linear frequency scale produced by the STFT does not align with human auditory perception, which is logarithmic. To optimize the neural network's feature space and discard imperceptible data, the linear spectrogram is projected onto the Mel-scale [37].

This transformation approximates human hearing. This is done thanks to applying series of overlapping triangular filters to the frequency bins:

$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right) \tag{2.2}$$

The frequency axis is typically compressed into a defined number of Mel-bands (e.g., Nmels = 80) as seen in Fig. 6.2. The filter bank is constrained to a specific frequency range (e.g., 0 Hz to 8000 Hz) to capture the primary characteristics and textures of environmental sound while filtering out imperceptible high-frequency noises.

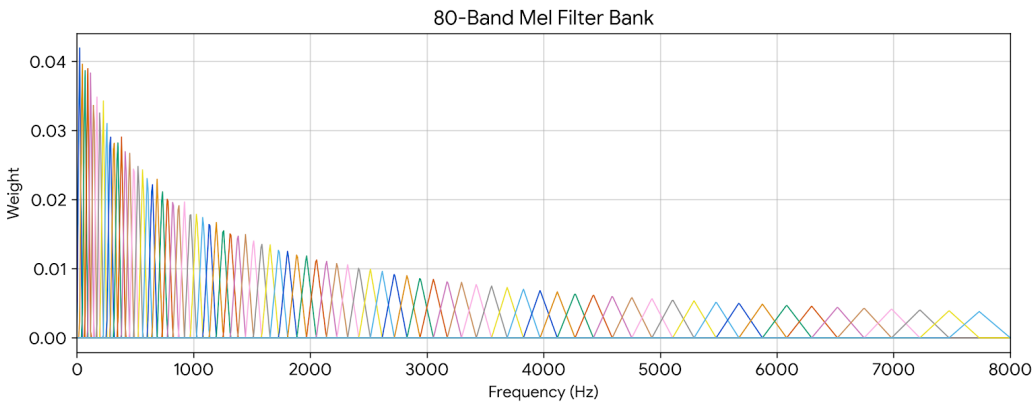

Figure 2.2: A graph illustrating the Mel filter bank

In neural audio literature, there is a noted trade-off regarding the number of bands. Andreu & Aylagas state that high-dimensional conditioners like 80-band spectrograms encourage high structural quality, whereas lower-dimensional representations (like 20 bands) often encourage greater diversity in the generated output [1]. However, preserving textural differences between surfaces (like metal and wood) generally requires the higher resolution of 80 bands, relying on the generative model's latent space to introduce diversity rather than reducing the spectrogram resolution.

2.1.4 Normalization and Log-Amplitude Representation

Deep neural networks, particularly Generative Adversarial Networks using Tanh activation functions in the generator, require strictly bounded input data to prevent vanishing or exploding gradients [13]. Audio data, has a massive dynamic range. To ensure successfull training of the GAN, the Mel-spectrogram must be normalized.

When the data is being prepared for trainong, the linear amplitude Mel-spectrogram (S_{amp}) is converted to a logarithmic decibel (dB) scale. This compresses the dynamic range, making the data distribution more manageable for neural networks:

$$S_{dB} = 20 \log_{10}(S_{amp}) \tag{2.3}$$

Following the logarithmic compression, statistical standardization (such as Z-score normalization) is commonly applied. By using the mean (μ) and standard deviation (σ) of the training dataset, the spectrograms are centered and scaled. To satisfy the constraints of bounded activation functions, this standardized data is often scaled by a constant factor and clamped to a strict range, such as [-1.0, 1.0]. This mathematical bounding ensures the dataset variance fits cleanly within the active region of the neural network without forceful clipping.

On the other hand, during inference, the spectrogram generated by the GAN

must be denormalized, before being passed on to the vocoder. Because many modern vocoders expect inputs in natural logarithmic amplitude space rather than standardized decibels, the generator's output must undergo an exact mathematical reversal:

1. Mapping the bounded data back to the original decibel scale using the dataset's (μ) and (σ).

2. Inverting the decibel equation to retrieve linear amplitudes.

3. Applying a natural logarithm to the amplitudes, often utilizing a small numerical floor (e.g., 10^5) to prevent undefined values during moments of absolute digital silence.

2.2 Machine Learning & Deep Generative Models

The sounds synthesis of artificial environmental sounds falls under the domain of deep generative modeling. Traditional discriminative models, such as classifiers that aim to predict a target label y given an input x. Unlike that, generative models aim to learn the underlying probability distribution $P_{data}(x)$ of the dataset itself [13]. Thanks to approximating this distribution, model can sample new data points. These data points are novel, but statistically speaking indistinguishable from the real training data.

To be able to process a 2D Mel-Spectrogram from Section 2.1.3, modern architectures for synthesis rely on deep neural networks that ane built using convolutional operations.

2.2.1 Foundations of Neural Networks

A fully connected neural network usually struggles with high dimensional image data. The cause is exponential explosion of parameters and loss of spatial topology. Convolutional Neural Networks (CNN) prevent this from happening by utilizing learnable kernels that slide across the input. They perform mathematical convolution operations to extract spatial hierarchies of features [25]. Applying that to an audio spectrogram, kernels like these learn to detect low level acoustic textures in early levels (e.g., noise of a snare drum). Then in deeper layers, the high level semantic structures (e.g., rhythmic cadence).

Important part of these networks is the non-linear activation function. Without non-linear activations, a deep neural network, no matter how many layers, would collapse into just a linear transformation. Two activation functions are widely utilized. Both of them should in generative audio pipelines stabilize the synthesis of features:

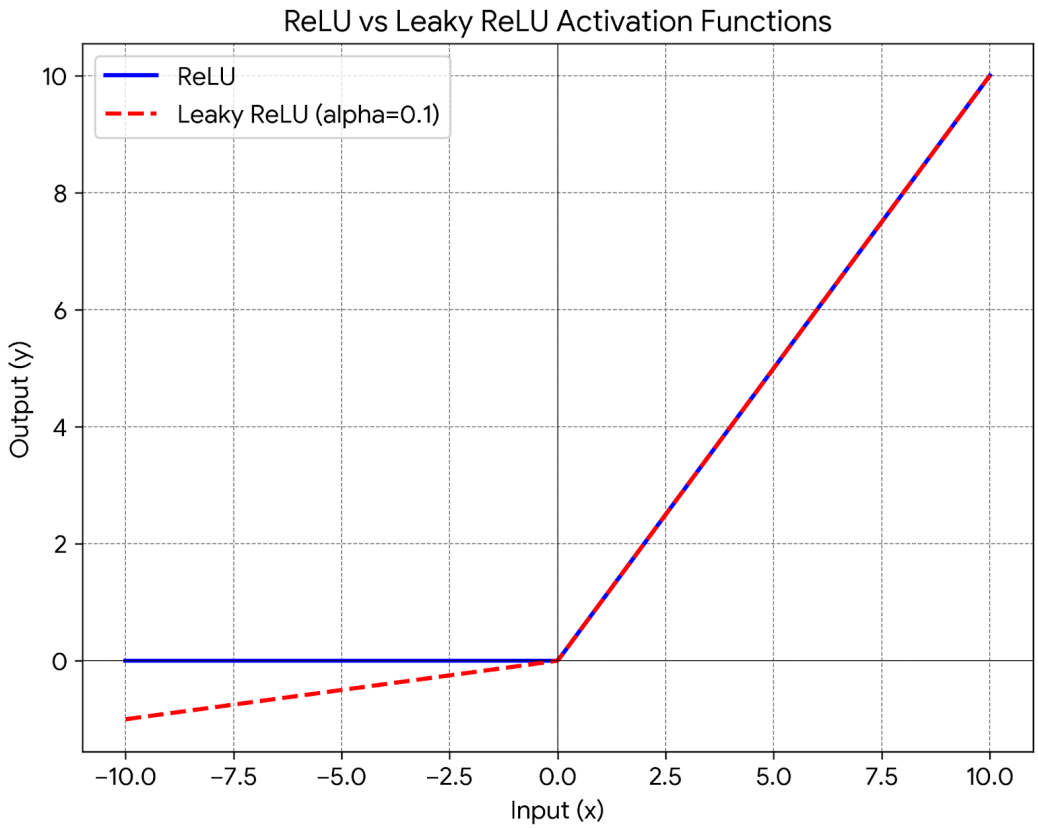

Figure 2.3: Rectified Linear Unit compared to Leaky Rectified Linear Unit

- **Leaky Rectified Linear Unit (LeakyReLU):** ReLU functions return zero for any input that is negative. This may cause neurons to die. This would cease updating during training. LeakyReLU, as the differs from that. As the name suggests it allows a small, non-zero gradient for negative values (e.g., $f(x) = \max(0.2x, x)$) 2.3. Thanks to this, LeakyReLU is widely used in generative architectures, as it ensures robust gradient flow during highly unstable adversarial training.

- **Hyperbolic Tangent (Tanh):** This function squashes any real value input into the symmetric range of $[-1.0, 1.0]$.

$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{2.4}$$

Usually, the Tanh function is explicitly applied at the final output layer of the generator. f the target Mel-spectrograms are normalized to $[-1.0, 1.0]$ during data preparation, the Tanh activation explicitly forces the generator's predictions to match the mathematical bounds of the real data, preventing extreme outliers and gradient explosions [34].

2.2.2 Taxonomy of Generative Architectures

In the present there are a few distinct families of generative models. There are many tradeoffs to be justified.

1. **Variational Autoencoders (VAEs)** Operating on the principle of encoding input data into a lower dimensional probabilistic latent space and reconstructing it via a decoder. They optimize the Evidence Lower Bound (ELBO) on the log-likelihood of the data [22]. VAEs offer stable training. On the contrast of that, they are prone to generating blurry outputs. This happens because their element-wise reconstruction loss penalizes high-frequency details. The sounds of footsteps on gravel or sand would lose the sharp transients and a critical textures, that are necessary for environmental audio.

2. **Flow-Based Models** This class of models learns an explicit, invertible mapping between the data distribution and a simple noise prior. As it was demonstrated by Andreu & Aylagas [1], flow-based models as their WaveFlow can generate high quality raw audio by optimizing maximum likelihood directly. The problem is that every layer must be mathematically invertible. This makes them intensive on memory and computationally expensive [1].

3. **Diffusion Probabilistic Models** Recently most go to models in generative fidelity [19]. Models as AudioLDM and AudioGen synthesize data by iteratively removing Gaussian noise from a signal over dozens or hundreds of steps [24][29]. Their iterative sampling process is very slow, but produces great sample quality.

For the interactive entertainment where real time generation is considered, procedural implementation, and rapid inference speed, are of high priority. Highly desirable are models that generate complex data in a single forward pass.

2.2.3 The Generative Adversarial Network Framework

To overcome blurry outputs of VAEs as well as the slow inference speed of diffusion models, many implementations use the Generative Adversarial Network (GAN) framework, introduced by Goodfellow et al. [14] [28]. GANs do not optimize a lower bound on likelihood and do not rely on definition of the probability density. They use a Minmax game for two. This concept utilizes a game theoretic approach, where two neural networks compete 2.4:

- Generator (G): The generator takes a random vector noise z from the last distribution (like $\mathcal{N}(0, I)$), then maps it to data space $\mathrm{G}(z)$. The goal of the generator is to synthesize artificial data indistinguishable from real recordings.

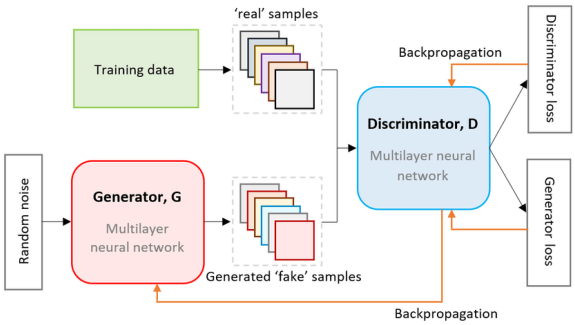

Figure 2.4: A classic GAN architecture diagram.

- Discriminator (D): Acts as a binary classifier. It analyzes the samples it receives and evaluates the probability, weather the data comes from the generator or the real dataset.

The training process is defined by the following value function $V(D, G)$ [14]:

$$\min_G \max_D V(D, G) = \mathbb{E}x \sim pdata(x)[\log D(x)] + \mathbb{E}z \sim pz(z)[\log(1 - D(G(z)))] \tag{2.5}$$

In audio synthesis context, adversarial loss acts as texture critic. In the case that the generator produces an audio spectrogram necessary high frequencies or is too smooth, discriminator can easily tell it is fake. This forces the generator to synthesize fine details that other architectures would smooth out due to just mathematics under the bonnet. Once the generator is trained, it produces audio in a single forward pass, outpacing diffusion or autoregressive models.

2.2.4 Advanced Architectural Components for Audio

Standard convolutional GANs were originally designed for synthesizing of visual images like animals or human faces. A GAN like this usually struggles with more complex audio. Normal photographs are translationally invariant. If a picture of a dog is shifted few pixels to some direction, it still remains a dog. Shifting a spectrogram on y-axis alters the pitch and the semantic meaning of the sound. This is addressed by these specialized neural components:

- Coordinate Convolutions: Standard CNNs are translationally invariant. This makes them to struggle to map specific frequency bands. Coordinate Convolutions (CoordConv) solve this by explicitly concatenating extra channels to the input tensor that represent the spatial x (time) and y (frequency) coordinates [30]. Thanks to this the convolutional filters and spatially aware.

- Self-Attention Mechanisms: CNNs capture local textures well, but because their receptive fields are limited, they struggle with long-range temporal dependencies, like keeping ambient reverb consistent over several seconds of audio. Self-attention, made popular by the Transformer architecture [38], computes a weighted matrix that describes how every feature in a sequence relates to every other feature. Adding self-attention to a convolutional GAN enforces global structural coherence across the whole temporal sequence, which substantially improves realism.

2.3 GAN Stability

Although the Generative Adversarial Network framework is a powerful approach to learning probability distributions, its original formulation is well known for its training instability. To understand the mathematical and architectural design decisions adopted in audio generation systems, it is first necessary to analyze the limitations of the standard GAN objective.

2.3.1 GAN Training

In the original GAN formulation[14]2.4, the Discriminator D is trained to be a binary classifier using the standard cross entropy loss. Setup like this has two common and well documented failures:

- Vanishing Gradient: If the discriminator becomes too good to quickly, meaning it can accurately distinguish real and fake audio in early stages of training, then the loss gradient starts approaching zero. This is a problem, because the generator then does not receive feedback on how to improve.

- Mode Collapse: This is in simple terms the opposite of a vanishing gradient. It happens when the generator discovers a few spectrograms that can fool the discriminator. This leads to the generator using the same sample all the time, which collapses diversity. The generator should be able to output different sounds for one textual input.

To tackle this issues, the standard cross entropy is usually replaced. Most common are better distance metrics and strict regularization applied to the network weights.

2.3.2 Hinge Loss

To counteract the vanishing gradient problem, conditional architectures use hinge loss instead of Minimax or Wasserstein loss [4]. The Hinge Loss imposes a strict confidence

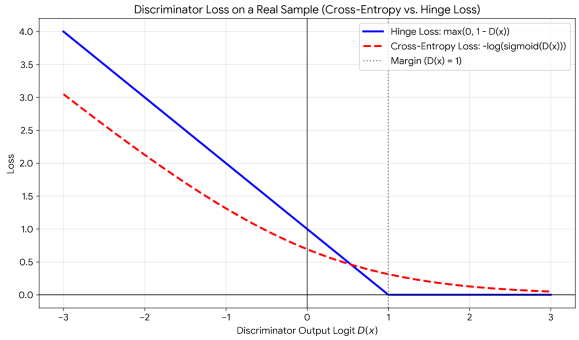

Figure 2.5: Hinge loss compared to cross entropy loss.

margin on the discriminator. The objective functions for the discriminator (L_D) and the generator (L_G) are defined as follows:

$$L_D = \mathbb{E}_{x\sim\mathbb{P}_r}[\max(0, 1 - D(x))] + \mathbb{E}z \sim \mathbb{P}_z[\max(0, 1 + D(G(z)))] \tag{2.6}$$

$$L_G = -\mathbb{E}_{z\sim\mathbb{P}_z}[D(G(z))] \tag{2.7}$$

This penalizes the discriminator only if it scores real sample less than 1.0, or a fake sample greater than -1.0. Once the discriminator pushes a real sample past the margin of 1 with confidence, the loss becomes zero 2.5. By preventing the discriminator from over-optimizing on easily classified samples, Hinge Loss ensures that continuous gradient flows back to the generator [27]. That makes the generator to improve acoustic fidelity.

2.3.3 Spectral Normalization

For functions as Hinge or Wasserstein to function correctly, he discriminator must behave as a smooth distance metric. The function must be Lipschitz continuous[1]. The Rate at which the discriminator's output changes in response to a change in input must be strictly bounded.

Older implementations of GANs tried to either forcefully clip network weights or to apply Gradient Penalties (WGAN-GP) [16]. Weight clipping is known reduce network's capacity to learn complex spectral features. Gradient penalties are computationally expensive.

More elegant solution is **Spectral Normalization**. This technique explicitly bounds the Lipschitz constant of the network by normalizing the weight matrix W

[1]Function f is L-Lipschitz continuous, if there exists a real constant $L \geq 0$ such that for all x_1, x_2 in the domain:

$$||f(x_1) - f(x_2)||_2 \leq L||x_1 - x_2||_2$$

. In GANs, L is typically constrained to 1 to bound gradients

of every convolutional and linear layer by its largest singular value [31]:

$$W_{SN} = \frac{W}{\sigma(W)} \tag{2.8}$$

Application of Spectral Norm only to the discriminator forces the network to remain 1-Lipschitz continuous. This stabilizes the training and does not restricting the complexity of the features it can extract. Ideal for highly complex, high-frequency environmental textures.

2.3.4 Feature Matching Loss

Hinge Loss and Spectral Normalization stabilize the mathematical gradients. **Feature Matching Loss** directly combats mode collapse and improves fidelity of generated audio. [36].

Usually generator receives only a single scalar value as feedback. That is the final Real/Fake score. Feature Matching changes this dynamic. As the discriminator analyzes a spectrogram, it extracts abstract representations at various depths of its network. Feature Matching forces the generator to synthesize audio that statistically matches the real audio at these intermediate hidden layers. Without this the generator could fool discriminator in just the final layer.

This is done by extracting the activations $f(x)$ from a specific layer of the discriminator and applying an L1[2] penalty between the features of the real and generated data:

$$L_{FM} = \mathbb{E}\left[||f(x) - f(G(z))||_1\right] \tag{2.9}$$

Thanks to this the generator is forced to match the structural statistics of real data. Something like a broadband noise distribution at layer 2, or the harmonic resonance at layer 4. Single mode collapse should not happen. Generated spectrograms should have multilayered textures necessary for believable environmental audio.

2.4 Natural Language Conditioning

Standard conditional architectures often rely on discrete class labels (e.g., $y \in \{0, 1, \ldots, K\}$).They treat "footstep on snow" and "footstep on gravel" as entirely distinct categories. This approach ignores semantic similarities and prevents the generation new combinations. To enable open ended generation, the model must project unstructured text into a continuous semantic vector space.

[2]Mean Absolute Error

2.4.1 Contrastive Language-Audio Pretraining (CLAP)

Models, such as BERT, can produce context aware text embeddings. The problem is that they have no understanding of how those words relate to acoustic properties. To bridge the gap between text and sound, Contrastive Language-Audio Pretraining (CLAP) can be utilized.

CLAP utilizes a dual encoder architecture:

- one Transformer-based encoder for text
- one specialized encoder for audio[3]

During pretraining, CLAP employs a contrastive learning objective[4]. This forces the text encoder and the audio encoder to project their respective inputs into same shared latent space [9].

Thanks to extracting text embedding from already pretrained CLAP model, the generative pipeline receives a dense vector that already has acoustic texture of the prompt. Semantically similar descriptions are clustered closely together. That allows the generator to synthesize accurate transients, even for descriptions with more than one word.

2.4.2 Conditional Group Normalization (CGN)

After the text prompt is encoded into a dense vector c, the generator injects this information into the image generation process. Simple concatenation at the input layer provides a baseline. More advanced architectures actively change the spatial features of the convolutional layers using Conditional Group Normalization.

Other normalization techniques stabilize the training, but can take away stylistic variations. CGN re-injects the text information. It is done by predicting scaling and shifting parameters of the normalized feature maps [7].

For the convolutional feature map x, Group Normalization is applied to standardize data first. Standard Batch Normalization normalizes across batch dimension and introduces dependencies between samples. Group Normalization divides the channels of a single sample into independent groups and normalizes spatial features. This normalization happen strictly in these groups. As this removes the batch dependency that can disrupt adversarial stability, it is highly advantageous for GANs.

The text embedding c is passed through a learned linear projection to output two modulation vectors: $\gamma(c)$ and $\beta(c)$ 2.6. Final conditioned feature map is computed as:

[3] such as HTS-AT

[4] Contrastive learning is a machine learning paradigm that optimizes the embedding space by maximizing cosine similarity between positive pairs, while minimizing the similarity between negative pairs.

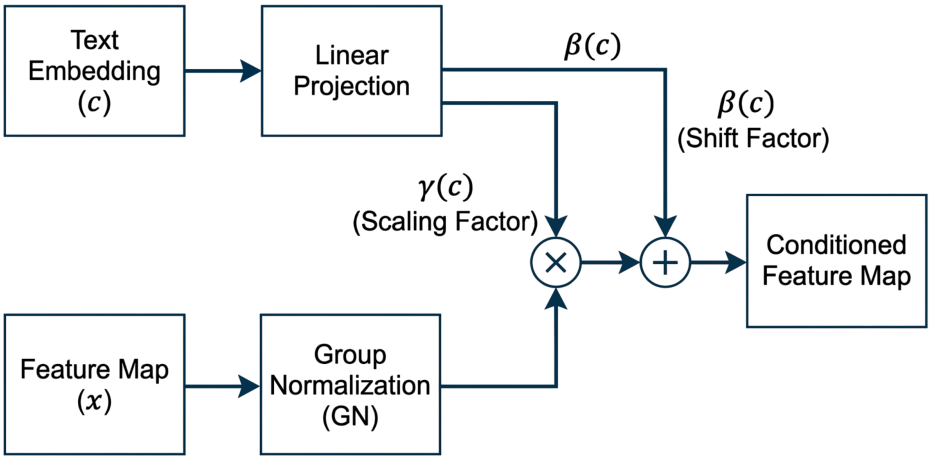

Figure 2.6: Architectural flow of CGN. Text embedding c is projected to predict the affine parameters γ and β. This modulates normalized audio feature maps.

$$\mathrm{CGN}(x, c) = \gamma(c) \cdot \mathrm{GN}(x) + \beta(c)$$

This allows the language prompt to act as a style controller at multiple resolutions throughout the generator network. Explicitly scaling up feature maps corresponding to target textures[5].

2.4.3 Projection Discriminator

The discriminator must not only evaluate if an audio spectrogram looks mathematically real. It also has to evaluate whether it is accurate with the provided text prompt. Projection Discriminator does not just concatenate text embedding with the spectrogram [32]. Discriminator acts as an energy based model that computes the alignment between the image and the text. The computation is done using an inner dot product.

Flattened, high dimensional feature vector $\phi(x)$ is extracted from the spectrogram. Parallel with that the text embedding c is projected this same dimensionality. Then the final pre activation output of discriminator $D(x, c)$ is defined as sum of the audio realism score and the text-audio alignment score:

$$D(x, c) = A(\phi(x)) + \phi(x)^T V(c)$$

A is a learned linear transformation evaluating unconditional realism. V is a projection matrix for text embedding. The inner product: $\phi(x)^T V(c)$, measures the cosine similarity between the generated audio features and the semantic intent of the text. If the audio sounds real but fails to match the prompt the discriminator's score is penalized. In theory forcing the generator to listen to user's text request.

[5]Boosting high-frequency noise channels when the prompt includes the word "sand".

2.5 Neural Vocoding

The generative pipeline ends by producing a high-fidelity Mel-spectrogram. Because this is a 2D magnitude-based representation, it cannot be processed by audio hardware. **Neural Vocoder** is required to perform phase reconstruction. It also maps these features back into a 1D time-domain waveform. This is a hard task since spectrogram is a loss projection. Therefore the vocoder must intelligently synthesize the missing temporal data.

2.5.1 Phase Reconstruction

The Mel-spectrogram is a magnitude only projection of the STFT. STFT of a signal x yields a complex number:

$$X(m, \omega) = |X(m, \omega)| \cdot e^{j\phi(m,\omega)} \tag{2.10}$$

$|X(m, \omega)|$ represents the magnitude spectrum. $\phi(m, \omega)$ represents the phase spectrum. This reduces the complexity of the generative task, but it results in a significant loss of phase information[6].

Historically, phase reconstruction was primarily handled by the Griffin-Lim algorithm. This algorithm is a iterative heuristic that alternates between time and frequency domain projections[15]. computationally lightweight, Griffin-Lim optimizes for mathematical consistency. This comes at a tradeoff of quality of the sound. This results quite frequently into temporal features not reconstructing well. The sounds come of as metallic, alien like artifacts.

2.5.2 Generative Vocoding

Modern audio synthesis bypasses the limits of manual strategies by using generative neural vocoders. These deep neural networks are trained to map Mel-spectrograms to high quality audio. It is done by learning the phase distribution from real data. A key architecture, HiFi-GAN[23], uses Multi Period Discriminators to reconstruct rhythmic structures of audio. These models were optimized for periodic nature of human speech. Due to this, they often fail to generalize to the chaotic noise profiles of environmental audio.

[6] data that defines the precise temporal alignment of frequencies

2.5.3 NVIDIA BigVGAN

NVIDIA BigVGAN is a neural vocoder, that can ensure high fidelity synthesis even for environmental classes. It is build upon the multi period discriminator structure of Hifi-GAN[23]. Building on this BigVGAN introduces architectural refinements. This allows the model to reconstruct non speech data[7] without requiring class specific retraining[26]. BigVGAN achieves this universal adaptability through two big modifications:

1. **Anti-Aliased Representation:** Standard vocoders tend to from aliasing artifacts in the upsampling process, especially in the parts, where high frequency noise blends into the audible spectrum. BigVGAN mitigates this by integrating low-pass filters before every sampling rate change. It strictly adheres Nyquist-Shannon sampling theorem. Model eliminates digital folding, allowing for the clear synthesis of environmental textures.

2. **Periodic Inductive Bias:** Standard activation functions[8] are inefficient at modeling the oscillatory nature of audio. BigVGAN addresses this by using the Anti-Aliased Representation, also called the Snake activation function:

$$f(x) = x + \frac{1}{a}\sin^2(ax) \tag{2.11}$$

   Snake activation incorporates a periodic sine signal directly into the neuron's output[39]. This architectural shift allows for more accurate reconstruction of resonant environmental sounds[9].

By conditioning BigVGAN on the structured 80-band Mel-spectrograms, the vocoder should function as artifact less phase estimator. It avoids robotic distortions common in simpler synthesis models. The result is a environment that maintains physical accuracy and perceptual quality.

2.6 Evaluation Metrics for Generative Models

Evaluating generative models like GANs is significantly more difficult than evaluating discriminative models. Discriminative tasks have clear labels for calculating error. Generative models aim to produce original data that follows a target distribution. Because of this metrics like Mean Squared Error are ineffective. Generated sound can be perceptually perfect even if it has high mathematical error. Evaluation must rely on comparing latent feature distributions and assessing human perceptual realism.

[7] such as Foley and environmental noise
[8] LeakyReLU
[9] metallic ringing or percussive wood impacts

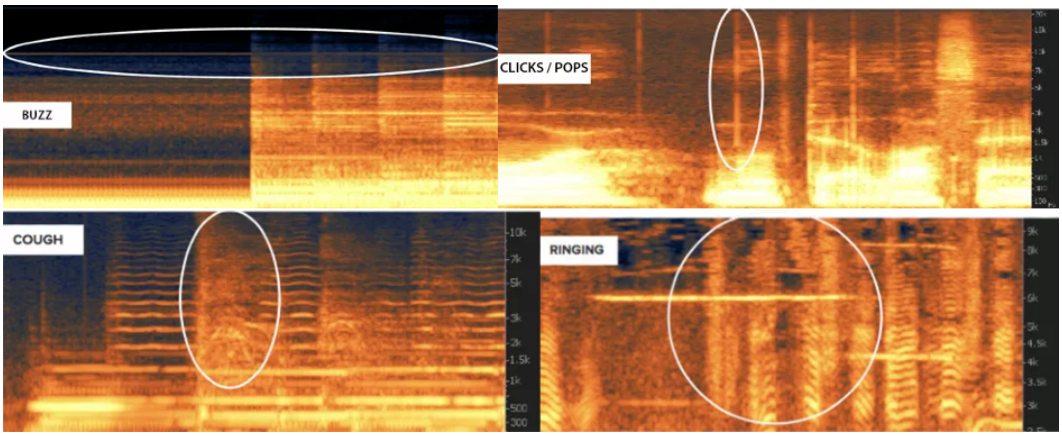

Figure 2.7: Enter Caption

2.6.1 From Inception Distance to Fréchet Audio Distance

For determining realism and diversity of generated samples, the Fréchet Inception Distance (FID) is used as the standard metric [18]. FID utilizes a pretrained Inception-v3 network to extract deep feature vectors from both real and synthetic datasets. It assumes these feature distributions can be modeled as multivariate Gaussians and calculates the Fréchet distance [10] between them:

$$d^2 = ||\mu_r - \mu_g||^2 + \mathrm{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}) \tag{2.12}$$

(μ_r, Σ_r) and (μ_g, Σ_g) represent the mean and covariance matrices of the real and generated feature distributions. Lower distance indicates that the generated data is statistically indistinguishable from the real data. But applying FID to audio spectrograms directly is not right. Inception network is trained on visual objects[11], not acoustic textures. Neural audio pipelines utilize the Fréchet Audio Distance (FAD)[21].

Fréchet Audio Distance (FAD) uses the same mathematical framework as FID. However, it replaces the visual Inception model with **VGGish**. That is a CNN pretrained on the AudioSet corpus for sound classification. FAD calculates distance between the latent feature distributions of real and synthesized audio. Then it evaluates whether they share consistent acoustic statistics and temporal structures. FAD compares entire datasets rather than individual pairs. Due to that it effectively identifies global issues like mode collapse or persistent distortion.

2.6.2 Subjective Evaluation

enter cap

Objective evaluation in audio synthesis is still an imperfect science. Statistical scores do not always correlate with how humans actually hear. A model could score

[10] Wasserstein-2 distance
[11] like dogs and cars

strong FAD score and still produce artifacts that a human would find distracting. Mean Opinion Score (MOS) remains the gold standard for evaluation. MOS test consists of human listeners rating generated samples on a 1–5 Likert scale[6]. This provides measurement of quality automated benchmarks often miss.

- Fidelity: Does the sample sound like a physical recording that is free from digital artifacts?
- Alignment: Does the sound accurately match the semantic intent of the text condition[12]?

The model's success depends on checking these perception goals with the target demographic.

[12] does heavy footstep on gravel sound heavy

Chapter 3

Related Work

Deep learning is redefining digital audio generation. To justify the custom Generative Adversarial Network a benchmark against current models is ideal. This chapter goes over dominant solutions in environmental audio. Three important metrics for evaluation are acoustic fidelity, generative diversity, and computational inference speed. This section reviews industry practices for audio synthesis and discusses their limitations.

3.1 Procedural Game Audio

Before neural synthesis can be analyzed baseline methodologies currently utilized in interactive digital environments should be established. Game engines have relied on two primary audio paradigms in the past. static sample playback and procedural Digital Signal Processing (DSP).

Most prevalent is playback of prerecorded audio samples. When an event occurs within a game[1], the game engine triggers a corresponding audio file. Repeatedly retrieving identical audio breaks the immersion. To mitigate this massive libraries of variations are used. Then in real time randomization of pitch and volume of these audio files takes place. It can create minor variations, but it is not possible to alter the acoustic texture like this. Especially for applications that center around audio more than visuals, the lack of dynamic diversity leads to cognitive fatigue and worsened navigation[12]. What does not help this approach even more is that storing thousands of high fidelity variations for every single sound category can drastically increase the size of the application.

Procedural generation can lower this memory constraint. It uses complex DSP graphs to create sounds in real time. Modern game engines have dedicated virtual machines for solely this purpose. Such example could be Unreal Engine's MetaSounds[11]. Meta sounds allows for sound designers to heavily parametrize sounds. By applying

[1] player stepping on specific surface

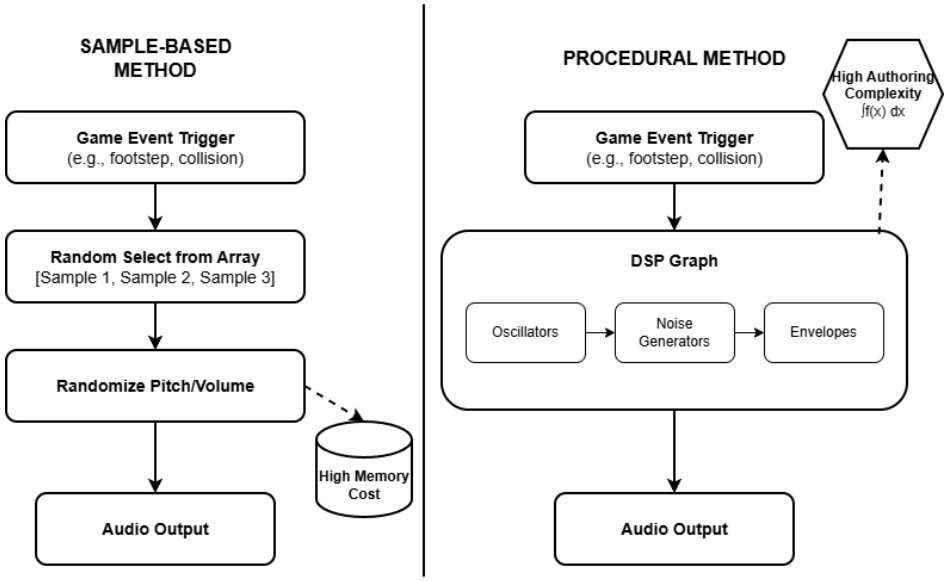

Figure 3.1: Comparison of sample-based methodologies with procedural methods

combining oscillators, noise generators, and filters sounds can be altered programmatically 3.1.

Procedural audio excels at real time creation of sound, and technically offers infinite variations by changing parameters, it is not all just great news. Designing a highly accurate procedural model of a complex environmental sound like random irregular crunch of gravel is very difficult. It can take whole team of experts that have knowledge of digital signal processing and physical acoustics. It is same as building music instrument from the scratch, not just playing one. Another downfall is that the DSP graphs cannot be controlled with text prompts. This makes the procedural generation very lack dynamics.

Limitations as mentioned above guide the current solutions towards deep generative modeling[6].

3.2 Autoregressive Audio Generation

Neural paradigm for text to audio synthesis treats audio generation as a sequence modeling problem. Autoregressive Audio Generation directly mirrors architecture of LLMs[2] used for language processing.

Raw audio waveforms contain tens of thousands of samples per second. It is computationally unmanageable for a Transformer network to predict raw audio sample by sample. Modern autoregressive architectures like Meta's AudioGen solve this by using neural audio compression[24].

AudioGen utilizes a neural audio codec to compress the audio waveform into a

[2]Large Language Model

discrete sequence of low dimensional acoustic tokens. Audio is discretized into vocabulary. Once this is done, Transformer language model is trained to predict the next audio token in the sequence, which is conditioned on a semantic text embedding of the prompt[24]. The sequence of tokens is predicted leading into decoder that reconstructs the tokens back into audio.

Autoregressive approach can produce high quality environmental sounds with great semantic alignment to textual prompts. However, the problem is the inference speed. This is due to the fact that the Transformer must predict each token sequentially, because the generation of token N strictly depends on the output of token $N-1$. This makes the generative process hard to be parallelized on a GPU.

Chapter 4

Work methodology and Research methods

Chapter 5

Implementation

5.1 Architecture Design

To tackle the primary objective of implementation, Deep Generative Model has been created. This model is capable of doing a synthesis of environmental sounds, such as footsteps on different surfaces. Generation of these sounds is conditioned to a natural language descriptions. Different from speech synthesis, where it is possible to rely on rigid structures for generation. In the task of creating environmental sound the model needs to be capable of capturing the randomness of these sounds, whilst still maintaining semantic coherence with the textual inputs.

To fulfill this objective, this, the system is built upon a Conditional Wasserstein Generative Adversarial Network with Gradient Penalty (cWGAN-GP). The standard GANs are proven, to be very instable when modeling high-dimensional spectral data [17]. This architecture was selected to solve that issue. The implementation leverages **PyTorch** to create **Generator (*G*)** and **Critic**[1] ***D*)**. These two neural networks technically compete against each other, where the Generator creates audio spectrograms from latent noise and text embeddings, and the Critic evaluates the realism of the generated content.

5.1.1 The General network (G)

The generator is the one responsible for mapping a text embedding t and a latent vector z into a Mel-spectrogram x_fake. The architectural design follows a transposed convolutional network[4] structure, effectively acting as a decoder that upsamples low-dimensional latent information into a high-resolution time-frequency representation.

Input Representation and Conditioning The generation process begins with two distinct inputs:

[1]Critic in a more standard GAN is also called the Discriminator.

1. **Text embedding t:** A vector of dimension d_t which has been derived from the sentence-transformer. Semantic intent of the prompt is encapsulated of this embedding.

2. **Latent vector z:** A noise vector of dimension $d_z = 100$ sampled from a standard normal distribution $N(0, 1)$. This component ensures the necessary stochasticity, thanks to which the model can generate diverse variations of the same sound class (e.g., different patterns for "footsteps") without messing-up the semantic meaning.

In the initial stage of the network, these two vectors are concatenated along the channel dimension, resulting in a combined input vector of size 484. This ensures that the entire generation process, from the very first projection, is conditioned on the textual description.

The combined vector is then projected via a fully connected linear layer to a dense representation of shape $(512, 5, 32)$. This tensor is then reshaped to match the format required for 2D convolution ($Channels \times Height \times Width$).
The core of the Generator consists of a stack of four **Transposed Convolutional blocks** `ConvTranspose2d`. Unlike standard interpolation, transposed convolutions contain learnable weights, which allows the network to learn optimal upsampling filters specific to spectral textures. The network then doubles the spatial resolution while halving the channel depth:

- upsamples to 1024 channels in block one
- progressively reduce channels to 256 and 128 blocks two and three
- finalize projection to 1 channel, creates single-channel spectrogram in block four

Rectified Linear Units `ReLU` are employed after every intermediate layer to introduce non-linearity and mitigate the vanishing gradient problem[2]. Crucially, the final output layer utilizes a Hyperbolic Tangent `Tanh` activation function. This function bounds the output values to the range $[-1, 1]$. Because of this, the data pipeline includes a strictly enforced normalization step to scale the logarithmic Mel-spectrograms, from decibels to this interval $[-1, 1]$. This leads to compatibility between the real data distribution and the generator's range.

5.1.2 The Critic Network (D) and Wasserstein Loss

Standard GANs typically use a Discriminator with a Sigmoid output to classify samples as "Real" (1) or "Fake" (0). However, this approach often leads to the vanishing

[2]When a highly accurate Discriminator produces zero gradients, leaving the Generator unable to learn

gradient problem, where the Generator stops learning if the Discriminator becomes too successful too quickly.

To address this issue, **Wasserstein GAN (WGAN)** is implemented. Instead of a binary classifier common for the GAN networks, the network acts as a Critic. It produces a continuous scalar score representing the "realness" of the input, approximating the Earth Mover's (Wasserstein) distance between the real and generated data distributions.

The normalization layers in the Critic were proven to be necessary during the implementation. At first the design utilized **Batch Normalization** `BatchNorm`, a standard technique in Deep Learning. However, theoretical constraints of the WGAN-GP framework dictate that the Critic must satisfy the **1-Lipschitz continuity constraint**. That means the gradient of the Critic must not explode. Batch Normalization creates dependencies between samples in a batch (using batch statistics), which invalidates the independence required for the gradient penalty calculation. The solution seems to be to use **Instance Normalization** `InstanceNorm2d` which normalizes each sample independently using its own statistics, preserving the independence of data points required for the valid computation of the Gradient Penalty.

The Critic mirrors the Generator's structure but in reverse. It utilizes strided 2D Convolutions *`Conv2d` with `stride=2`* to progressively downsample the input spectrogram.

- Activation: LeakyReLU with a negative slope of 0.2 is used. Unlike standard ReLU, LeakyReLU allows a small gradient to flow even for negative values. This helps to prevent "dead neurons" in the Critic [3].

- Late Fusion of Text: Oppposite from the Generator (which fuses text at the start), the Critic injects the text embedding after the convolutional feature extraction. The spectral features are flattened and concatenated with the text embedding before the final dense classification layers. This forces the Critic to first learn clean audio features before evaluating whether those features match the text.

5.1.3 Stabilization via Gradient Penalty

The the Lipschitz constraint that has been mentioned above needs to be strictly enforced. To achieve this the **Gradient Penalty** term is used. It replaces weight clipping method from classic WGAN[2]. Weight clipping quite often leads to weight clustering at the clipping limits which is unwanted behavior. The Gradient Penalty is computed during the training loop by sampling random points along the straight lines between real and fake data pairs.The gradient of the Critic with respect to these interpolated

[3] common failure mode in GAN training

points is calculated, and the loss function is penalized if the norm of this gradient deviates from 1.

The total loss function L for the Critic is thus defined as:

$$L_{critic} = \underbrace{\mathop{\mathbb{E}}_{\tilde{x}\sim\mathbb{P}_g}[D(\tilde{x},t)] - \mathop{\mathbb{E}}_{x\sim\mathbb{P}_r}[D(x,t)]}_{\text{Wasserstein Estimate}} + \underbrace{\lambda \mathop{\mathbb{E}}_{\hat{x}\sim\mathbb{P}_{\hat{x}}}\left[(\|\nabla_{\hat{x}} D(\hat{x},t)\|_2 - 1)^2\right]}_{\text{Gradient Penalty}} \tag{5.1}$$

- $\tilde{x} \sim \mathbb{P}_g$: This represents the Fake Data `fakeSpec` in `train.py`. It is sampled from the generator distribution
- $x \sim \mathbb{P}_r$: This represents the Real Data `realSpec` in `train.p`.
- $\hat{x} \sim \mathbb{P}_{\hat{x}}$: This is the Interpolated Data `interpolated` in `train.py line 52`. It is the random point along the line between the real and fake samples where the gradient is penalized
- t: This is the Text Embedding `textEmb` in `train.py`.
- λ: The penalty coefficient `LAMBDAGP = 20` in `train.py`

Where in the solution λ is set to 20, a hyperparameter that weights the importance of the continuity constraint.

Chapter 6

Research

Chapter 7

Results and Discussion

Conclusion

Bibliography

[1] S. Andreu and M. Villanueva Aylagas. Neural synthesis of sound effects using flow-based deep generative models. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 18, pages 2–9, 2022.

[2] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *International conference on machine learning*, pages 214–223. PMLR, 2017.

[3] Adrián Barahona-Ríos and Tom Collins. Specsingan: Sound effect variation synthesis using single-image gans. *CoRR*, abs/2110.07311, 2021.

[4] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale gan training for high fidelity natural image synthesis, 2019.

[5] Matheus Cezarotto, Pamela Martinez, and Barbara Chamberlin. Developing inclusive games: Design frameworks for accessibility and diversity. In Branislav Sobota, editor, *Game Theory - From Idea to Practice*, chapter 3. IntechOpen, London, 2022.

[6] Mateo Cámara, Fernando Marcos, Anders R. Bargum, Cumhur Erkut, Joshua Reiss, and José Luis Blanco. Neural audio synthesis for sound effects: A scope review. *IEEE Transactions on Audio, Speech and Language Processing*, 2025.

[7] Harm de Vries, Florian Strub, Jérémie Mary, Hugo Larochelle, Olivier Pietquin, and Aaron C. Courville. Modulating early visual processing by language. *CoRR*, abs/1707.00683, 2017.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

[9] Benjamin Elizalde, Soham Deshmukh, Mahmoud Al Ismail, and Huaming Wang. Clap: Learning audio concepts from natural language supervision, 2022.

[10] Jesse Engel, Kumar Krishna Agrawal, Shuo Chen, Ishaan Gulrajani, Chris Donahue, and Adam Roberts. GANSynth: Adversarial neural audio synthesis. In *International Conference on Learning Representations*, 2019.

[11] Epic Games. Metasounds in unreal engine, n.d.

[12] Jonas Friberg and Dan Gärdenfors. Audio games: new perspectives on game audio. In *Proceedings of the 2004 ACM SIGCHI International Conference on Advances in computer entertainment technology*, pages 148–154. ACM, 2004.

[13] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. `http://www.deeplearningbook.org`.

[14] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks, 2014.

[15] D. Griffin and Jae Lim. Signal estimation from modified short-time fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 32(2):236–243, 1984.

[16] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein gans, 2017.

[17] Andrew Hennessy, Kenneth Clarke, and Megan Lewis. Generative adversarial network synthesis of hyperspectral vegetation data. *Remote Sensing*, 13(12):2243, 2021.

[18] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, Günter Klambauer, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a nash equilibrium. *CoRR*, abs/1706.08500, 2017.

[19] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020.

[20] Tero Karras, Miika Aittala, Samuli Laine, Erik Härkönen, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Alias-free generative adversarial networks, 2021.

[21] Kevin Kilgour, Mauricio Zuluaga, Dominik Roblek, and Matthew Sharifi. Fr\'echet audio distance: A metric for evaluating music enhancement algorithms. *arXiv preprint arXiv:1812.08466*, 2018.

[22] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[23] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. Hifi-gan: Generative adversarial networks for efficient and high fidelity speech synthesis, 2020.

[24] Felix Kreuk, Gabriel Synnaeve, Adam Polyak, Uriel Singer, Alexandre Défossez, Jade Copet, Devi Parikh, Yaniv Taigman, and Yossi Adi. Audiogen: Textually guided audio generation, 2023.

[25] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

[26] {Sang Gil} Lee, Wei Ping, Boris Ginsburg, Bryan Catanzaro, and Sungroh Yoon. Bigvgan: A universal neural vocoder with large-scale training. 2023. Publisher Copyright: © 2023 11th International Conference on Learning Representations, ICLR 2023. All rights reserved.; 11th International Conference on Learning Representations, ICLR 2023 ; Conference date: 01-05-2023 Through 05-05-2023.

[27] Jae Hyun Lim and J. C. Ye. Geometric gan. *ArXiv*, abs/1705.02894, 2017.

[28] Claire Little, Mark Elliot, Richard Allmendinger, and Sahel Samani. Generative adversarial networks for synthetic data generation: A comparative study, 12 2021.

[29] Haohe Liu, Zehua Chen, Yi Yuan, Xinhao Mei, Xubo Liu, Danilo Mandic, Wenwu Wang, and Mark D. Plumbley. Audioldm: Text-to-audio generation with latent diffusion models, 2023.

[30] Rosanne Liu, Joel Lehman, Piero Molino, Felipe Petroski Such, Eric Frank, Alex Sergeev, and Jason Yosinski. An intriguing failing of convolutional neural networks and the coordconv solution. *CoRR*, abs/1807.03247, 2018.

[31] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. *CoRR*, abs/1802.05957, 2018.

[32] Takeru Miyato and Masanori Koyama. cgans with projection discriminator. *CoRR*, abs/1802.05637, 2018.

[33] A.V. Oppenheim and R.W. Schafer. *Discrete-time Signal Processing*. Prentice-Hall signal processing series. Pearson, 2010.

[34] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015.

[35] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks, 2019.

[36] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, Xi Chen, and Xi Chen. Improved techniques for training gans. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016.

[37] S. S. Stevens, John E. Volkmann, and Edwin B. Newman. A scale for the measurement of the psychological magnitude pitch. *Journal of the Acoustical Society of America*, 8:185–190, 1937.

[38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[39] Liu Ziyin, Tilman Hartwig, and Masahito Ueda. Neural networks fail to learn periodic functions and how to fix it. *CoRR*, abs/2006.08195, 2020.